Gloria Paita - Web Developer 2024/2026

JavaScript's This FOR DUMMIES®

Learn to:

Finally convince yourself that THIS isn't personal, it's just JavaScript.

A Reference for the Rest of Us!®

INTRODUCTION

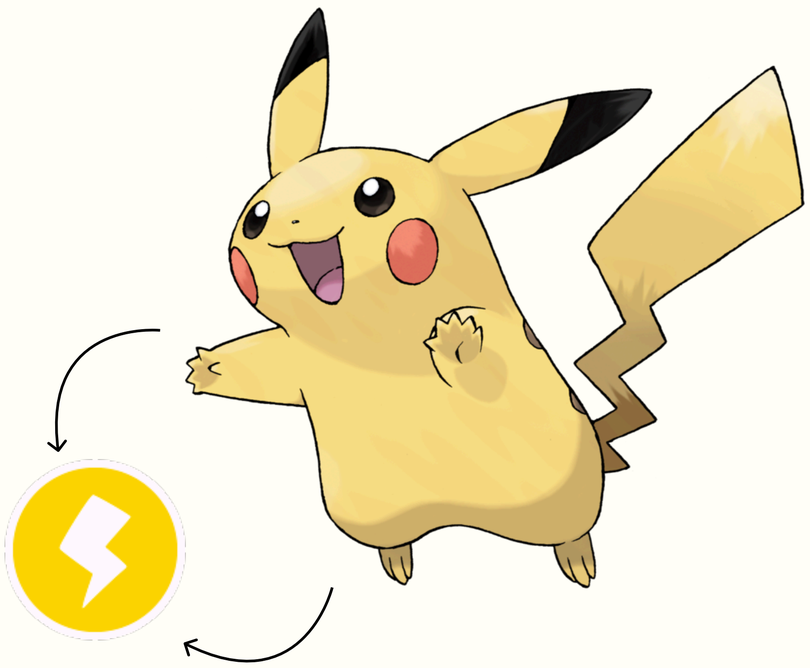
If you're a Web Dev student practicing JS, chances are it happened to you too:
you write a simple function, you call it with confidence and suddenly...

script.js ×

```
1   const pikachu = {
2     type: 'electric',
3     pokedexEntry: function () {
4       console.log('Pikachu is an ' + this.type + ' type Pokémon.');
5     },
6   };
7
8   pikachu.pokedexEntry();
9
```

Console ×

```
Pikachu is an electric type Pokémon.
```



INTRODUCTION

script.js

```
1  const pikachu = {
2    type: 'electric',
3    pokedexEntry: function () {
4      console.log('Pikachu is an ' + this.type + ' type Pokémon.');
5    },
6  };
7
8  const askPokedex = pikachu.pokedexEntry;
9  askPokedex();
```

Console

```
Pikachu is an undefined type Pokémon.
```

...BOOM!

Your THIS isn't what you thought it was!

script.js

```
1  const pikachu = {
2    type: 'electric',
3    pokedexEntry: function () {
4      console.log('Pikachu is an ' + this.type + ' type Pokémon.');
5    },
6  };
7
8  window.type = "window";
9
10 const askPokedex = pikachu.pokedexEntry;
11 askPokedex();
```

Console

```
Pikachu is an window type Pokémon.
```

One minute it's pointing to your **object**, the next it's... **undefined**? Or worse, the global **window** object?!

INTRODUCTION

...BOOM! Your THIS isn't what you thought it was.

One minute it's pointing to your object, the next it's... ...defined? Or worse, the global window object?!

script.js

```
1   const pikachu = {
2     type: 'electric',
3     pokedexEntry: function () {
4       console.log('Pikachu is an ' + this.type + ' type Pokémon.');
5     },
6   };
7
8   const askPokedex = pikachu.pokedexEntry;
9   askPokedex();
```

Console

```
Pikachu is an undefined type Pokémon.
```

t.js

```
1   const pik
2     t
3                y: function () {
4       nsole.log('Pikachu                is.                kémon.');
5     },
6   };

    ndow.type = "window";

10  const askPo                achu.pokedexEntry;
11  askPokedex()
```

Console

```
Pikachu is an window type Pokémon.
```

DEFINITION

SO, WHAT IS THIS?

this is a keyword meant to be a **reference to the context** where a function is supposed to run (where it is called).

It can be typically encountered when dealing with **Object Methods**, where it refers to the object that the method is attached to.
It's pretty cool, because it allows the same method to be reused on different objects!

script.js

```
1   class ElectricPokemon {
2     constructor(name) {
3       this.name = name;
4     }
5
6     thunderShock() {
7       console.log(`${this.name} used ThunderShock! ⚡`);
8     }
9   }
10
11  // Creating new Pokémon
12  const pikachu = new ElectricPokemon("Pikachu");
13  const zapdos = new ElectricPokemon("Zapdos");
14
15  // Calling the shared, inherited method
16  pikachu.thunderShock();
17  zapdos.thunderShock();
```

Console

```
Pikachu used ThunderShock! ⚡
Zapdos used ThunderShock! ⚡
```

THE BORING, UNFASHIONABLE QUESTION.

BUT WAIT!!

Everybody is always asking “**What’s this?**”
But nobody ever asks the most important question...

THE RIGHT QUESTION!

AN EVEN BETTER QUESTION!

"**Where** is this?"

"**How** is the function invoked?"

OUTSIDE OF FUNCTIONS

The value of this in JavaScript depends on **how a function is invoked** (runtime binding), not how it is defined.

But what if this is used outside of any function scope? What if we're still in the global execution context?

Then, this will equal to the **global object**

Global object

The global object is determined by the execution environment:

- Browser → **window object**
- Node.js → **global object**

```
> console.log(this);
                                                        VM653:1
  ▸ Window {window: Window, self: Window, document: document, name: '', location: Locatio
    n, …}
< undefined
```

INVOCATION TYPES

THE VALUE OF THIS IN JAVASCRIPT DEPENDS ON HOW A FUNCTION IS INVOKED.

When it comes to functions,
JavaScript has four function invocation types:

Function Invocation

script.js

```
function wildEncounter() {
  console.log("A wild Pokémon appeared! 👾");
}

wildEncounter();

```

Console

```
A wild Pokémon appeared! 👾
```

Method Invocation

script.js

```
const pikachu = {
  name: "Pikachu",
  attack: function () {
    console.log(`${this.name} used Thunderbolt!`);
  }
};

pikachu.attack();

```

Console

```
Pikachu used Thunderbolt!
```

Constructor Invocation

script.js

```
function Pokemon(name) {
  this.name = name;
  this.battle = function () {
    console.log(`${this.name} is ready to battle!`);
  };
}

const pikachu = new Pokemon("Pikachu");
pikachu.battle();

```

Console

```
Pikachu is ready to battle!
```

Indirect Invocation

script.js

```
function usePotion() {
  console.log(`${this.name} recovered HP!`);
}

const pikachu = { name: "Pikachu" };

usePotion.call(pikachu);

```

Console

```
Pikachu recovered HP!
```

STRICT OR SLOPPY?

PLEASE NOTE:

this behaves very differently in **strict mode** compared to non-strict ("**sloppy**") mode JavaScript runs on by default.

Strict mode can be applied by writing `"use strict";` at the top of:

- An **entire script**
- A **single function**

And will be active not only in the current scope but also in the inner scopes, for all functions declared inside.

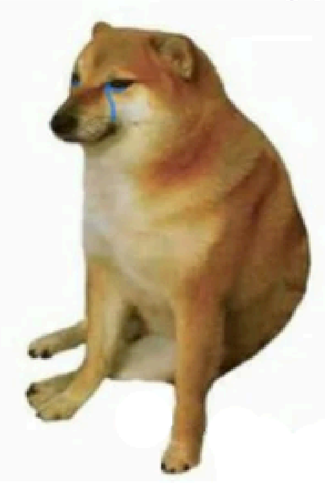
STRICT OR SLOPPY?

Strict Mode

- was born with ECMAScript 5.1
- activated using keyword → “use strict”;
- buff enough to throw all type of errors
- will protect your code
- already applied to classes and modules
- etc...

They can coexist
in the same script

“Sloppy Mode”



*unofficial nickname for
Non-Strict Mode

- the default
- doesn’t complain about errors
- distracted; allows:
  - accidental creation of global variables
  - duplicate parameter names
  - etc...

FUNCTION INVOCATION

A REGULAR FUNCTION INVOKED AS A STANDALONE FUNCTION

Sloppy Mode:

→ this refers to the **global object**

It doesn't matter how many nested functions there are, this will always refer to the global object when they are invoked this way!

script.js

```
1   function findFood(){
2       console.log("Look at that cake by the " + this);
3
4       function hungry(){
5           console.log("Can't wait to eat that " + this + " !");
6
7           function love() {
8               console.log("Aah, I love " + this + " so much!");
9           };
10
11          love();
12      };
13
14      hungry();
15  };
16
17  findFood();
```

Console

```
Look at that cake by the [object Window]
Can't wait to eat that [object Window] !
Aah, I love [object Window] so much!
```

FUNCTION INVOCATION

A REGULAR FUNCTION INVOKED AS A STANDALONE FUNCTION

Strict Mode:

→ this is **undefined**

JS won't automatically bind this to the global object.


```
> "use strict";

  function lostFood(){
      console.log("I swear there wasn't an " + this + " here...");

      function disappointed(){
          console.log("I can't eat " + this + " !");

          function hate() {
              console.log("Aah, I hate " + this + " so much!");
          };

          hate();
      };

      disappointed();
  };

  lostFood();


  I swear there wasn't an undefined here...
  I can't eat undefined !
  Aah, I hate undefined so much!
```

METHOD INVOCATION

METHOD: A FUNCTION STORED IN A PROPERTY OF AN OBJECT

A Method Invocation needs one of the following **property accessor forms** to call the function:

- **bracked notation** → goku['fight']()
- **dot notation** → vegeta.fight()

script.js

```
1   class Saiyan {
2       constructor(name) {
3           this.name = name;
4       }
5
6       getName() {
7           console.log("I'm " + this.name + ", the strongest warrior on Earth!");
8       }
9   };
10
11  const goku = new Saiyan('Goku');
12  goku['getName']();
```

Console

```
I'm Goku, the strongest warrior on Earth!
```

script.js

```
1   function Saiyan(name) {
2       this.name = name;
3   }
4
5   Saiyan.prototype.getName = function() {
6       console.log("I'm " + this.name + ", prince of all Saiyans!");
7   };
8
9   const vegeta = new Saiyan('Vegeta');
10  vegeta.getName();
```

Console

```
I'm Vegeta, prince of all Saiyans!
```

this is the **object that owns the method**

METHOD + FUNCTION INVOCATION

BE CAREFUL!!

A function invocation nested inside a method has a different context than its outer function!

script.js

```
1   const goku = {
2       name: "Goku",
3
4       superSaiyanMode: function () {
5           // this --> the goku object
6           console.log(`Within ${this}, ${this.name} turned blond!`);
7
8           function superDuperSaiyanMode() {
9             // this --> window (or undefined in Strict Mode)
10              console.log(`Within ${this}, ${this.name} turned EVEN MORE blond!`);
11          }
12
13          return superDuperSaiyanMode();
14      }
15  };
16
17  goku.superSaiyanMode(); // --> throws TypeError in strict mode
```

Console

```
Within [object Object], Goku turned blond!
Within [object Window],  turned EVEN MORE blond!
```

METHOD + FUNCTION INVOCATION

BE CAREFUL!!

A function invocation nested inside a method has a different context than its outer function!

script.js

```
1   const goku = {
2       name: "Goku",
3
4       superSaiyanMode: function () {
5           // this --> the goku object
6           console.log(`Within ${this}, ${this.name} turned blond!`);
7
8           function superDuperSaiyanMode() {
9             // this --> window (or undefined in Strict Mode)
10              console.log(`Within ${this}, ${this.name} turned EVEN MORE blond!`);
11          }
12
13          return superDuperSaiyanMode();
14      }
15  };
16
17  goku.superSaiyanMode(); // --> throws TypeError in strict mode
```

Console

```
Within [object Object], Goku turned blond!
Within [object Window],  turned EVEN MORE blond!
```

this → goku

METHOD + FUNCTION INVOCATION

BE CAREFUL!!

A function invocation nested inside a method has a different context than its outer function!

script.js

```
1   const goku = {
2       name: "Goku",
3
4       superSaiyanMode: function () {
5           // this --> the goku object
6           console.log(`Within ${this}, ${this.name} turned blond!`);
7
8           function superDuperSaiyanMode() {
9             // this --> window (or undefined in Strict Mode)
10              console.log(`Within ${this}, ${this.name} turned EVEN MORE blond!`);
11          }
12
13          return superDuperSaiyanMode();
14      }
15  };
16
17  goku.superSaiyanMode(); // --> throws TypeError in strict mode
```

Console

```
Within [object Object], Goku turned blond!
Within [object Window],  turned EVEN MORE blond!
```

this → global obj
(or undefined)

METHOD + FUNCTION INVOCATION

SOLUTION 1: INDIRECT INVOCATION

With **Indirect Invocations** (call(), apply(), bind()) one can manually set the context for this.

script.js

```
1   const goku = {
2     name: "Goku",
3
4     superSaiyanMode: function () {
5       // this -> the goku object
6       console.log(`Within ${this}, ${this.name} turned blond!`);
7
8       function superDuperSaiyanMode() {
9         // this -> the goku object
10        console.log(`Within ${this}, ${this.name} turned EVEN MORE blond!`);
11      }
12
13      // Indirect Invocation with .call()
14      return superDuperSaiyanMode.call(this);
15    }
16  };
17
18  goku.superSaiyanMode();
```

Console

```
Within [object Object], Goku turned blond!
Within [object Object], Goku turned EVEN MORE blond!
```

this → goku obj

METHOD + FUNCTION INVOCATION

SOLUTION 1: INDIRECT INVOCATION

With **Indirect Invocations** (call(), apply(), bind()) one can manually set the context for this.

script.js

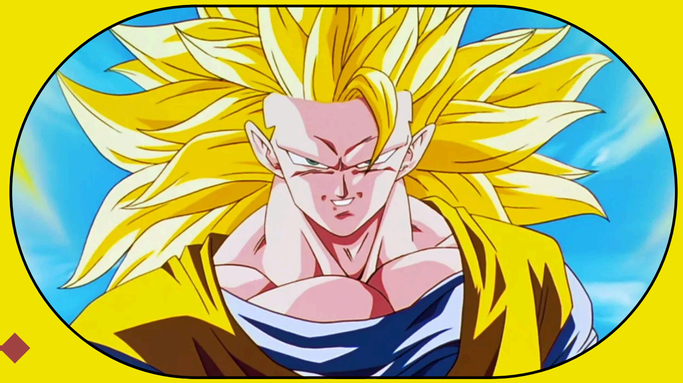
```
1   const goku = {
2     name: "Goku",
3
4     superSaiyanMode: function () {
5       // this -> the goku object
6       console.log(`Within ${this}, ${this.name} turned blond!`);
7
8       function superDuperSaiyanMode() {
9         // this -> the goku object
10        console.log(`Within ${this}, ${this.name} turned EVEN MORE blond!`);
11      }
12
13      // Indirect Invocation with .call()
14      return superDuperSaiyanMode.call(this);
15    }
16  };
17
18  goku.superSaiyanMode();
```

Console

```
Within [object Object], Goku turned blond!
Within [object Object], Goku turned EVEN MORE blond!
```

this → goku obj

this → goku obj



METHOD + FUNCTION INVOCATION

SOLUTION 2: ARROW FUNCTION

Arrow functions resolve this **lexically**: in this case, it uses the this value of goku.superSaiyanMode().

script.js

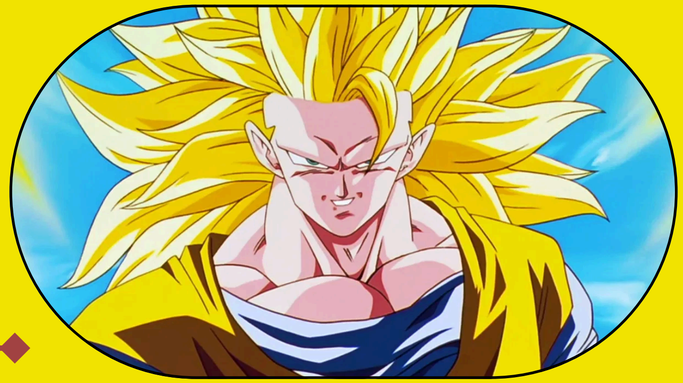
```
1   const goku = {
2     name: "Goku",
3
4     superSaiyanMode: function () {
5       // this -> the goku object
6       console.log(`Within ${this}, ${this.name} turned blond!`);
7
8       // this -> resolved lexically!
9       const superDuperSaiyanMode = () => {
10        console.log(`Within ${this}, ${this.name} turned EVEN MORE blond!`);
11      }
12
13      return superDuperSaiyanMode();
14    }
15  };
16
17  goku.superSaiyanMode();
```

Console

```
Within [object Object], Goku turned blond!
Within [object Object], Goku turned EVEN MORE blond!
```

this → goku obj

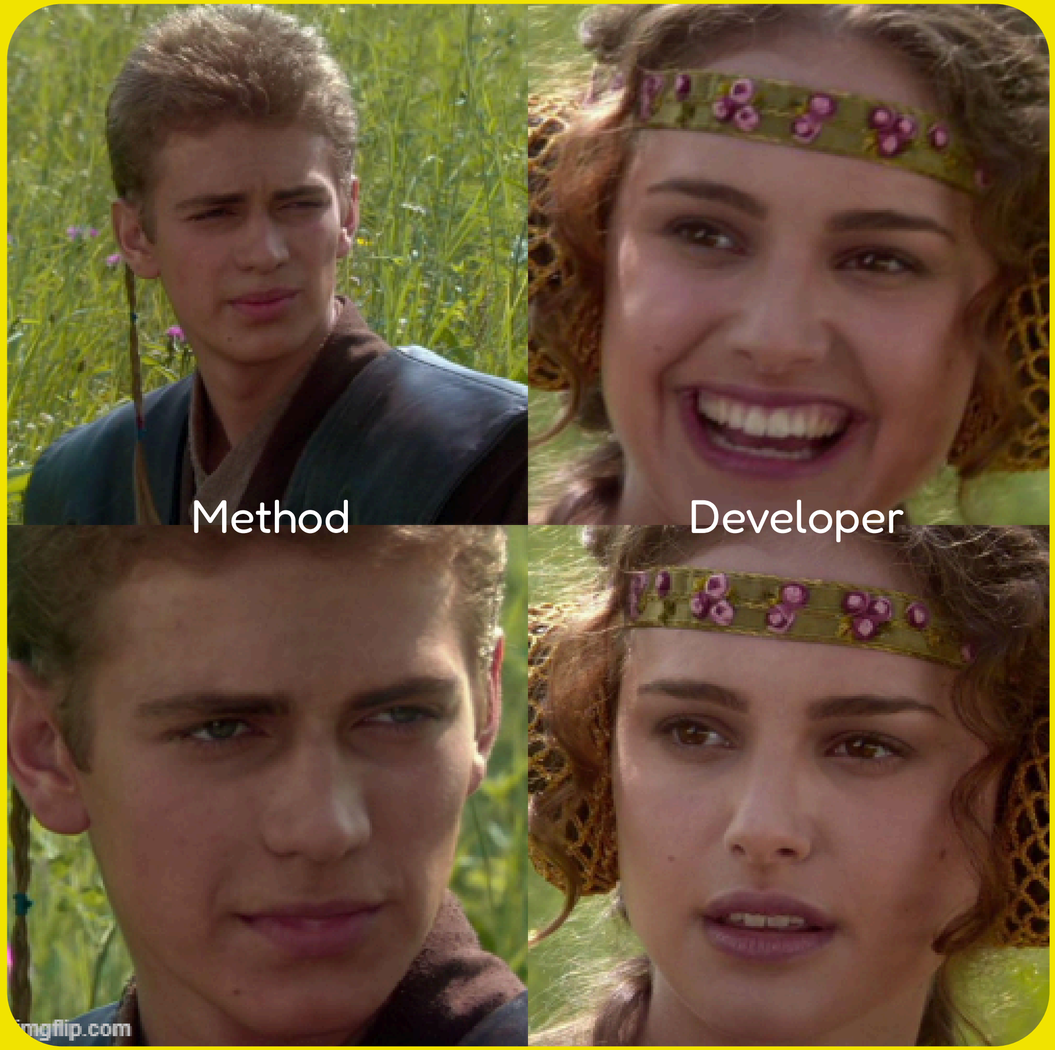
this → goku obj



SINGLING OUT THE METHOD

BE CAREFUL!!

"I'm a method, but you can also extract me from my object and use me separatedly."



"Amazing! And your **this** will still point to your object's context, right?"

...right?

SINGLING OUT THE METHOD

A METHOD IS SEPARATED FROM ITS OBJECT WHEN...

script.js

```
1   function PlotTwist(father, son) {
2     this.father = father;
3     this.son = son;
4
5     this.famousQuote = function () {
6       console.log(`${this.father}: ${this.son}, I am your father!`);
7     }
8   }
9
10  const starWars = new PlotTwist("Darth Vader", "Luke");
11
```

Method Invocation

```
13    starWars.famousQuote();
```

Console

```
Darth Vader: Luke, I am your father!
```

Extracting the method to a variable

```
13    const extractedFamousQuote = starWars.famousQuote;
14    extractedFamousQuote();
```

Console

```
undefined: undefined, I am your father!
```

Method as a parameter

```
13    setTimeout(starWars.famousQuote, 2000);
```

Console

```
undefined: undefined, I am your father!
```

A method called without an object is equivalent to a function invocation!
(this === **global/window** object or **undefined** in strict mode)

SINGLING OUT THE METHOD

SOLUTION 1: BINDING THIS WITH .BIND()

The **.bind()** method binds a function to an object.

script.js

```
1   function PlotTwist(father, son) {
2     this.father = father;
3     this.son = son;
4
5     this.famousQuote = function () {
6       console.log(`${this.father}: ${this.son}, I am your father!`);
7     }
8   }
9
10  const starWars = new PlotTwist("Darth Vader", "Luke");
11
13  const boundFamousQuote = starWars.famousQuote.bind(starWars);
```

Bound Function Invocation

```
15  //Bound Fn Invocation
16  boundFamousQuote();
```

Console

```
Darth Vader: Luke, I am your father!
```

Bound Function as a parameter

```
18  //Bound Fn as parameter
19  setTimeout(boundFamousQuote, 2000);
```

Console

```
Darth Vader: Luke, I am your father!
```

SINGLING OUT THE METHOD

SOLUTION 2: ARROW FUNCTION

script.js

```
1   function PlotTwist(father, son) {
2     this.father = father;
3     this.son = son;
4
5     this.famousQuote = () => {
6       console.log(`${this.father}: ${this.son}, I am your father!`);
7     }
8   }
9
10  const starWars = new PlotTwist("Darth Vader", "Luke");
11
```

Arrow functions resolve this **lexically**

Extracting the method to a variable

```
13  const extractedFamousQuote = starWars.famousQuote;
14  extractedFamousQuote();
```

Console

```
Darth Vader: Luke, I am your father!
```

Method as a parameter

```
13  setTimeout(starWars.famousQuote, 2000);
```

Console

```
Darth Vader: Luke, I am your father!
```

CONSTRUCTOR INVOCATION

USED TO CREATE A NEW OBJECT THAT INHERITS PROPERTIES FROM THE CONSTRUCTOR'S PROTOTYPE

script.js

```
1   function Dessert(type, ingredient) {
2       this.type = type;
3       this.ingredient = ingredient;
4       return this;
5   }
6
7   const cake = new Dessert("Cake", " Lie");
8   console.log(`The ${cake.type} is a ${cake.ingredient}.`);
9
```

Console

```
The Cake is a  Lie.
```

Constructor invocation is performed when **new** keyword is followed by a **constructor function**

this is the **newly created object!**

or cake in this case...

CONSTRUCTOR INVOCATION

DON'T FORGET THE "NEW" KEYWORD!!

script.js

```
1   function Dessert(type, ingredient) {
2       this.type = type;
3       this.ingredient = ingredient;
4       return this;
5   }
6
7   const cake = Dessert("Cake", "Lie");
8   console.log(`The ${cake.type} is a ${cake.ingredient}.`);
```

Console

```
The Cake is a Lie.
```

Some functions can create objects even by simply invoking them (without using the **new** keyword)...

CONSTRUCTOR INVOCATION

DON'T FORGET THE "NEW" KEYWORD!!

script.js

```
1   function Dessert(type, ingredient) {
2       this.type = type;
3       this.ingredient = ingredient;
4       return this;
5   }
6
7   const cake = Dessert("Cake", "Lie");
8   console.log(`The ${cake.type} is a ${cake.ingredient}.`);
9   console.log(cake === window);
10
```

Console

```
The Cake is a Lie.
true
```

Some functions can create objects even by simply invoking them (without using the **new** keyword)...

...right?

CONSTRUCTOR INVOCATION

DON'T FORGET THE "NEW" KEYWORD!!

script.js

```
1   function Dessert(type, ingredient) {
2       this.type = type;
3       this.ingredient = ingredient;
4       return this;
5   }
6
7   const cake = Dessert("Cake", "Lie");
8   console.log(`The ${cake.type} is a ${cake.ingredient}.`);
9   console.log(cake === window);
10
```

Console

```
The Cake is a Lie.
true
```

By performing a Function Invocation:

- you're setting **window** (or the global object, or undefined in strict mode) as the context for this.

- The properties are not added to a cake object, but directly to **window**.
- The **cake** object is not even created!

CONSTRUCTOR INVOCATION

DON'T FORGET THE "NEW" KEYWORD!!

script.js

```
1   function Dessert(type, ingredient) {
2       this.type = type;
3       this.ingredient = ingredient;
4       return this;
5   }
6
7   const cake = new Dessert("Cake", "Lie");
8   console.log(`The ${cake.type} is a ${cake.ingredient}.`);
9   console.log(cake === window);
10
```

Console

```
The Cake is a Lie.
false
```

Remember to use the **new** operator whenever you're planning on creating new objects!

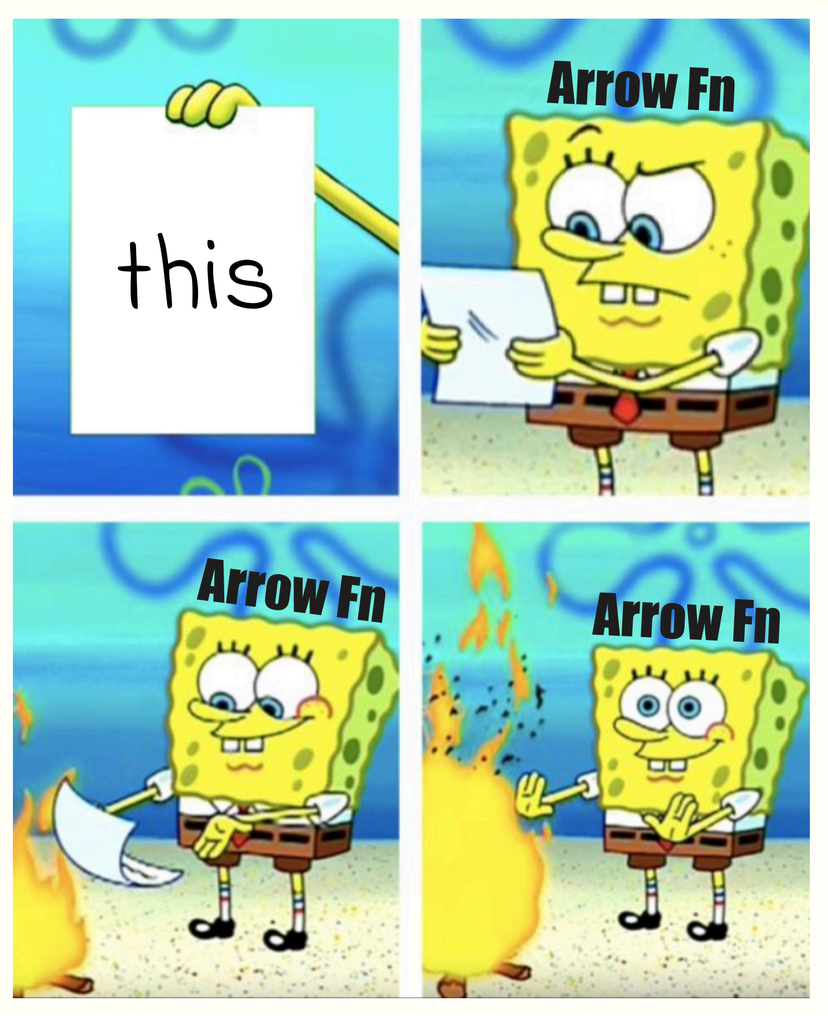
ARROW FUNCTION

A LIGHT, SHORTER SYNTAX FOR FUNCTION DECLARATION

Arrow functions are an exception to the rule.
Instead of creating their own execution context, **they inherit from the enclosing context at the time they are defined**, which is useful for callbacks and preserving context.

It means that:

- they **don't have their own this binding**
- their this is **lexically bound**.
- their this **cannot be modified or set** by Indirect Invocations



ARROW FUNCTION

ARROW FUNCTIONS AS OBJECT METHODS... #UNMETHODICAL

Arrow Function

script.js

```
1   function PastaDish(pastaType, sauce) {
2       this.pastaType = pastaType;
3       this.sauce = sauce;
4   }
5
6   PastaDish.prototype.dress = () => {
7       console.log("Here is your " + this.pastaType + " with " + this.sauce + "!");
8   };
9
10  const abhorrentDish = new PastaDish("Spaghetti", "Ketchup");
11  abhorrentDish.dress();
```

Console

```
Here is your undefined with undefined!
```

this === window
this !== current object

An arrow function gets its context at **definition** time.

Function Invocation

script.js

```
1   function PastaDish(pastaType, sauce) {
2     this.pastaType = pastaType;
3     this.sauce = sauce;
4   }
5
6   PastaDish.prototype.dress = function () {
7     console.log("Here is your " + this.pastaType + " with " + this.sauce + "!");
8   };
9
10  const abhorrentDish = new PastaDish("Spaghetti", "Ketchup");
11  abhorrentDish.dress();
```

Console

```
Here is your Spaghetti with Ketchup!
```

this !== window
this === current object

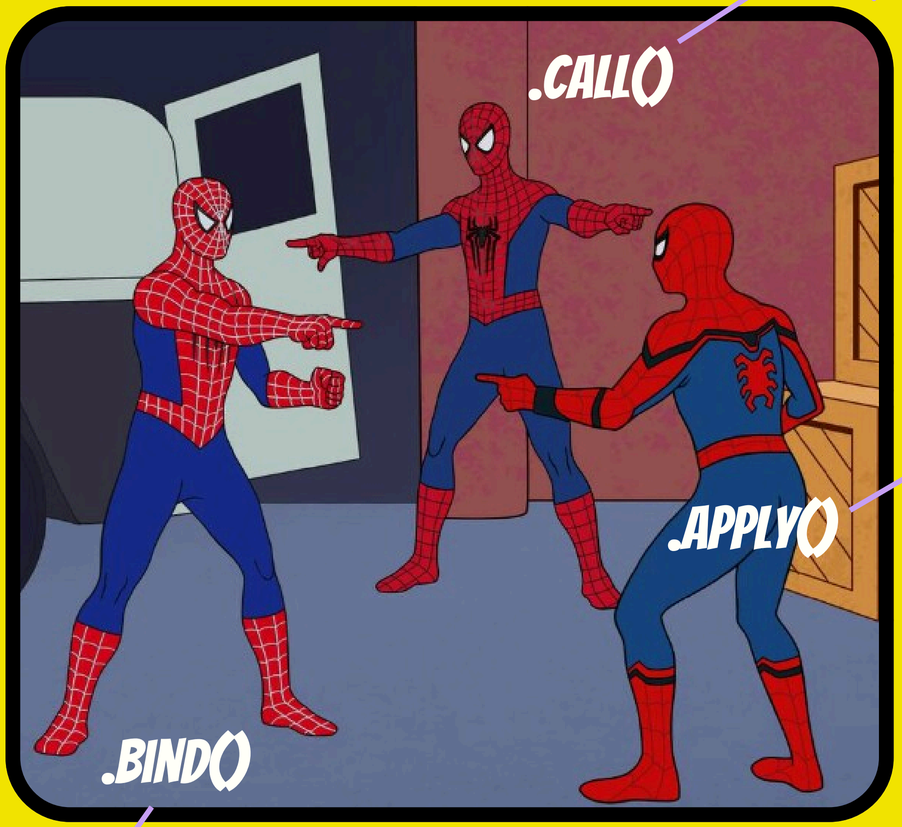
Regular functions change their context depending on **invocation**

INDIRECT INVOCATION

A WAY TO CONTROL WHICH CONTEXT (THIS) THE FUNCTION RUNS

The function is invoked via a method (call, apply, or bind), not by directly using ()

These are methods of Function instances (Function.prototype.foo())



Indirect & Immediate

Calls function immediately with this as the 1st argument and the other arguments passed individually

Indirect & Immediate

Same as .call(),
but arguments are passed as an array/array-like object.

Indirect & Deferred

Returns a copy of the function with a bound “this” (1st arg) and a bound list of args (optional).

A bound function cannot change its linked context when using .call(), .apply(), or .bind().

CONCLUSION

THE VALUE OF THIS:

Outside of a function

```
> console.log(this);
                                                    VM119:1
  Window {0: Window, window: Window, self: Window,
  ▸ document: document, name: '', location: Locatio
    n, …}
< undefined
>
```

- global object (Node.js)
- window object (browsers)

Function Invocation

script.js ×

```
1  function wildEncounter() {
2    console.log("A wild Pokémon appeared! 👾");
3  }
4
5  wildEncounter();
```

- global object (sloppy mode)
- undefined (strict mode)

Arrow Function

```
function PastaDish(pastaType, sauce) {
    this.pastaType = pastaType;
    this.sauce = sauce;
}

PastaDish.prototype.dress = () => {
    console.log("Here is your " + this.pastaType + " with " + this.sauce + "!");
};

const abhorrentDish = new PastaDish("Spaghetti", "Ketchup");
abhorrentDish.dress();
```

- inherit this from the parent scope
- their this cannot be changed or set

Method Invocation

script.js ×

```
1  const pikachu = {
2    name: "Pikachu",
3    attack: function () {
4      console.log(`${this.name} used Thunderbolt!`);
5    }
6  };
7
8  pikachu.attack();
```

- points to the method's object

Constructor Invocation

script.js ×

```
1  function Pokemon(name) {
2    this.name = name;
3    this.battle = function () {
4      console.log(`${this.name} is ready to battle!`);
5    };
6  }
7
8  const pikachu = new Pokemon("Pikachu");
9  pikachu.battle();
```

- points to the newly created object

Indirect Invocation

script.js ×

```
1  function usePotion() {
2    console.log(`${this.name} recovered HP!`);
3  }
4
5  const pikachu = { name: "Pikachu" };
6
7  usePotion.call(pikachu);
```

- .call() & .apply() set the this value (and params) for a particular call
- .bind() creates a bound function

THE END!

HAVE A GOOD DAY~

Now open VS Code
and make me proud!

(And thanks for listening!)